Hydro

Lettuce hydroponic system: how it works, what has been built, and how to run it

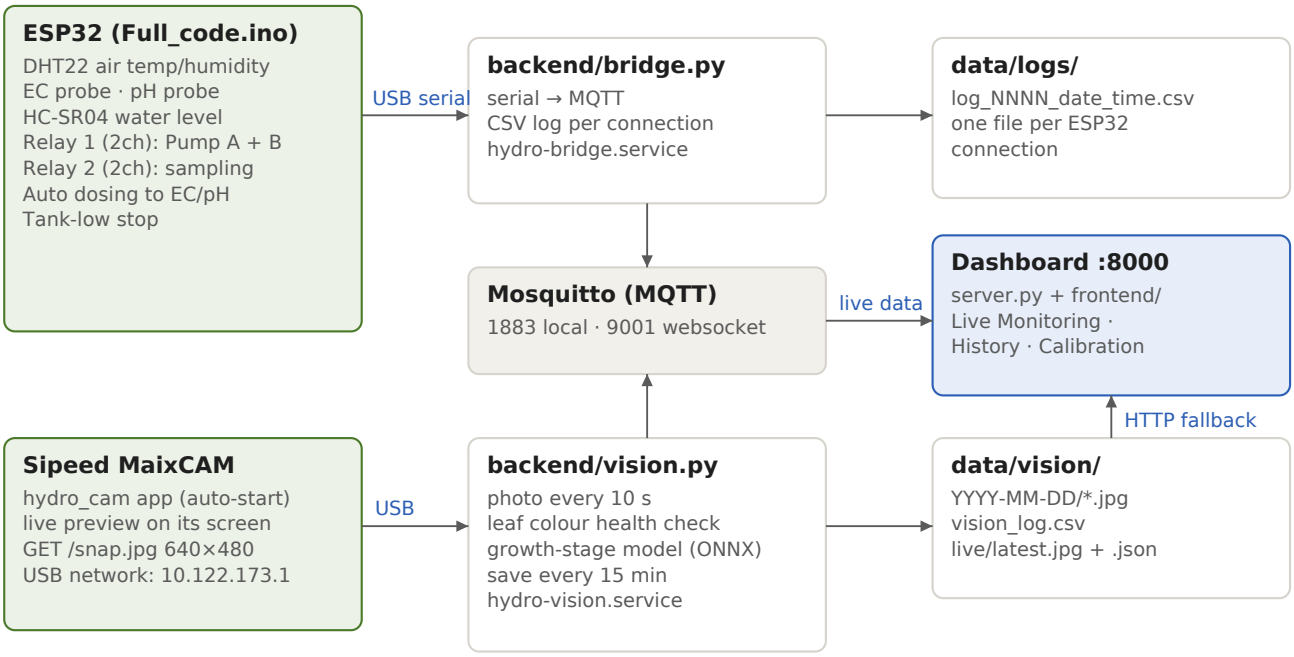
Raspberry Pi 5 · ESP32 · Sipeed MaixCAM · MQTT · Python
Version 4 · Status as of 29 September 2026 · Repository: github.com/muaz8172/hydro (private)

Contents	
1. System overview	What each part does and how data flows
2. Hardware and wiring	Pins, dividers, the two 2-channel relays, camera link
3. How it works	Sensors, pump control, logging, camera, AI model, dashboard, dataset
4. What has been done	Progress, commits and current status
5. How to run the system	First-time setup, daily use, commands
6. Calibration	EC, pH and tank level; target ranges
7. Troubleshooting	Common problems and fixes
8. Known issues and next steps	Safety items and roadmap

What's new in version 4
• **Minimalist dashboard** titled Hydroponic System Monitoring, with tabs Live Monitoring, Sensor History and Calibration, and DM Sans + Fraunces fonts (stored offline). The camera frame is square and shows the same centre crop the stage model sees (the dataset is 224×224). Sensor tiles show a green dot or a short tag (LOW, HIGH, DRIFT); raw values move to hover tooltips. New Plant vision card (canopy with change vs yesterday, yellowing, green index); a small 24 h graph follows the tile you click; each sensor has its own colour on Sensor History. See section 3.6.
• Version 3: **New plant dataset:** 88 lettuce photos, balanced across the three stages (seedling 30, vegetative 29, matured 29), all resized to 224×224 for training. Before: 41 photos with only 6 vegetative.
• **Growth-stage model retrained** (`backend/models/stage.onnx`): 86 % test accuracy, vegetative recall 79 % (target raised from 50 % to 70 %). See section 3.5.
• Version 2 changes are still in place: two 2-channel relays (section 2.1), air temperature target 25–36 °C (`temp_ok`), the three dashboard tabs, and the `frontend/`, `backend/`, `firmware/` layout.

1. System overview

The system grows lettuce in a hydroponic tank and watches it all day without internet. An **ESP32** reads the water and air sensors and switches the pumps through two 2-channel relays. A **Raspberry Pi 5** collects everything, stores it, and serves a dashboard. A **Sipeed MaixCAM** camera photographs the plants so their health, growth stage and size can be checked.



Everything talks through **MQTT** (the Mosquitto broker on the Pi). Each program publishes on its own topic, and the dashboard listens to all of them live.

MQTT topic	Sent by	Contents
`hydro/sensors`	bridge.py	One JSON reading every 2 s from the ESP32, plus `log_id` and `row` that match the CSV line
`hydro/log`	bridge.py	Text messages from the ESP32 (calibration, pump ON/OFF, tank low)
`hydro/cmd`	you / dashboard	Commands forwarded to the ESP32 (e.g. `status`, `ph7`, `pumpauto`, `sampleoff`)
`hydro/vision`	vision.py	Plant result (JSON) every 10 s: health, stage, size
`hydro/vision/image`	vision.py	The latest camera photo (JPEG, retained)
`hydro/vision/status`	vision.py	`online` or `offline: reason`
`hydro/vision/cmd`	dashboard	`snap` = save a photo to disk now

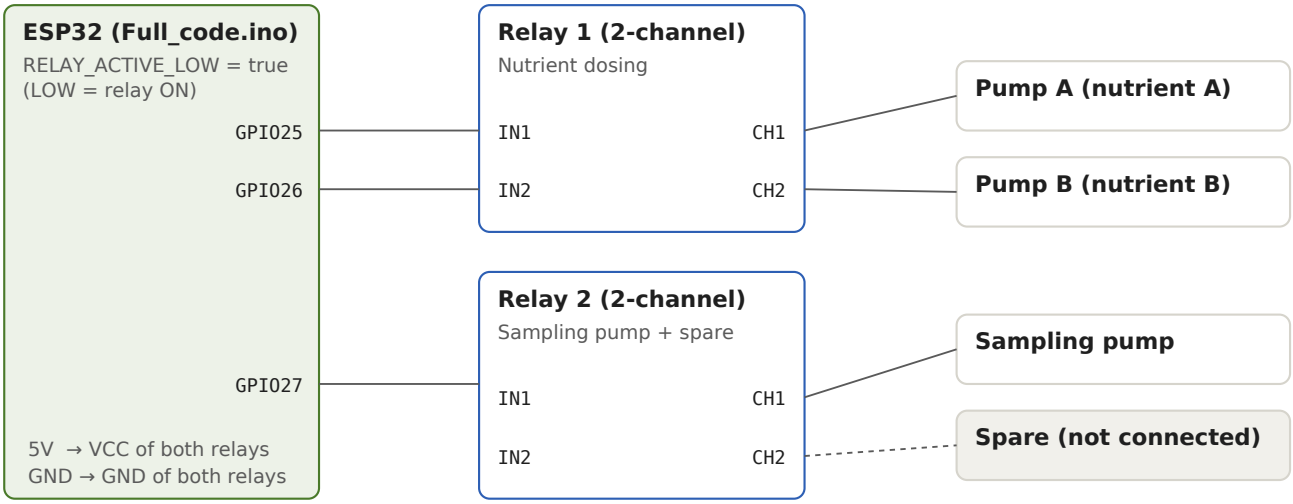
2. Hardware and wiring

Part	Pins	ESP32 / Pi	Notes
DHT22 (air temp / humidity)	DATA / VCC / GND	GPIO15 / 3.3 V / GND	10 k pull-up if the module has none
EC sensor (EC V2.1)	A0 / VCC / GND	GPIO34 / 3.3 V / GND	T1 unconnected; patched DFRobot_EC library
pH sensor (PH-4502C)	Po / V+ / G	GPIO35 via divider / 5 V / GND	10 k / 20 k divider is required
HC-SR04 (water level)	TRIG / ECHO / VCC / GND	GPIO32 / GPIO33 via divider / 5 V / GND	1 k / 2 k divider on ECHO; mounted above the tank, facing down
Relay 1 (2-channel)	IN1 / IN2 / VCC / GND	GPIO25 (Pump A) / GPIO26 (Pump B) / 5 V / GND	Active LOW; GND shared with the ESP32
Relay 2 (2-channel)	IN1 / IN2 / VCC / GND	GPIO27 (sampling pump) / not connected / 5 V / GND	Active LOW; IN2 is a spare channel; GND shared with the ESP32
ESP32 → Pi	USB	/dev/esp32 (udev rule)	115200 baud, one JSON line every 2 s
MaixCAM → Pi	USB-C	10.122.173.1 (camera), 10.122.173.100 (Pi)	Appears as a USB network adapter; the Pi gets its address automatically

Never put more than 3.3 V on an ESP32 pin. The pH module and the HC-SR04 run on 5 V, which is why their outputs go through voltage dividers. Keep the pH and EC probes in separate containers when testing; they interfere with each other.

2.1 Relays: two 2-channel modules

The pumps are switched by **two identical 2-channel relay modules**. The old 1-channel relay for the sampling pump has been removed. Relay 1 doses the nutrients; relay 2 runs the sampling pump and keeps one channel free.



Relay	Channel	ESP32 pin	Drives	Firmware name	JSON field
Relay 1	IN1	GPIO25	Pump A (nutrient A)	`PUMP_A_PIN`	`pump_a`
Relay 1	IN2	GPIO26	Pump B (nutrient B)	`PUMP_B_PIN`	`pump_b`
Relay 2	IN1	GPIO27	Sampling pump	`SAMPLE_PIN`	`pump_sample`
Relay 2	IN2	—	Spare, not connected	—	—

- Both modules share one setting in `Full_code.ino`: `RELAY_ACTIVE_LOW true` (LOW = relay ON). If your modules turn ON with HIGH, set it to `false`. The old separate `SAMPLE_ACTIVE_LOW` setting is gone.
- Relay VCC goes to the ESP32's 5 V, and relay GND must be shared with the ESP32 GND, otherwise the IN signals have no reference.
- At boot every relay pin is set to OFF before it becomes an output, so no pump kicks on while the ESP32 starts. The sampling pump switches on with the first reading (about 2 s after start-up).
- The spare channel (relay 2 IN2) is free for a future grow-light relay (see section 8). It is not used by the firmware yet.

> **Mains safety.** If a pump or a future grow light runs on mains voltage, keep the relay contacts and mains wiring in a closed box, away from the water, and use a fused supply.

3. How it works

3.1 Sensors (ESP32)

Every 2 seconds the ESP32 reads all sensors and prints one JSON line over USB:

```
{"temp":28.1,"hum":70.2,"ec":1.45,"ec_low":false,"ph":6.20,"level":9.8,"level_pct":72,
 "level_low":false,"ec_mv":237,"po_mv":2650,"dist_cm":10.2,
 "pump_a":true,"pump_b":true,"pump_auto":true,"pump_sample":true,"temp_ok":true}
```

Field	Meaning
`temp, hum`	Air temperature (°C) and humidity (%) from the DHT22
`ec`	Electrical conductivity (mS/cm): how much nutrient is in the water
`ec_low`	true = below the ESP32's measuring floor (about 0.6 mS/cm), e.g. plain water
`ph`	pH value of the nutrient solution
`level, level_pct`	Water height (cm) and tank level (%) from the HC-SR04, after tank calibration
`level_low`	true = tank below 20 %
`pump_a, pump_b`	Relay 1 IN1 / IN2 state (true = running)
`pump_sample`	Relay 2 IN1 state: the sampling pump (true = running)
`pump_auto`	true = automatic EC/pH control, false = manual commands
`temp_ok`	true = air temperature inside 25–36 °C, null = no DHT22 reading. Only reported; nothing heats or cools
`ec_mv, po_mv, dist_cm`	Raw sensor values, used for calibration and debugging

3.2 Pump control (ESP32)

The firmware keeps the nutrient solution inside the lettuce target: EC 1.2–1.8 mS/cm and pH 5.5–6.5.

- **Auto mode** (on at boot): Pump A and Pump B (relay 1) run together while EC or pH is outside the target, and stop when both are inside. They start again if either drifts out. `ec_low` counts as under target.
- **Sampling pump** (relay 2 IN1): runs all the time to move water past the sensors. It is not affected by EC/pH. `sampleoff` stops it (e.g. for maintenance), `sampleon` starts it again.
- **Tank-low protection**: when the tank is below 20 % (after tank calibration) all three pumps, on both relays, are forced OFF, in auto and manual, so they never run dry. `pumpaon`, `pumpbon` and `sampleon` are refused until the tank is refilled.
- **Manual**: `pumpaon`, `pumpaoff`, `pumpbon`, `pumpboff` switch to manual; `pumpauto` goes back to auto.
- A pH reading that is not a number stops Pumps A and B for safety.
- Air temperature is only checked against 25–36 °C and reported as `temp_ok`. No relay acts on it.

Situation	Pump A + Pump B (relay 1)	Sampling pump (relay 2)
EC or pH outside target (auto)	ON	ON
EC 1.2–1.8 and pH 5.5–6.5 (auto)	OFF (target reached)	ON
pH reading invalid (auto)	OFF (safety)	ON
Tank low (calibrated tank)	OFF, also in manual	OFF
`sampleoff` sent	unchanged	OFF until `sampleon`

3.3 Data logging (Raspberry Pi, backend/bridge.py)

`bridge.py` reads the ESP32's serial port, publishes each reading to MQTT and writes it to a CSV file. Each time the ESP32 connects (plugged in, Pi reboot, bridge restart) a new file starts:

```
data/logs/log_0007_2026-09-23_07-28-38.csv
log_id,row,time,temp,hum,ec,ec_low,ph,level,level_pct,level_low,dist_cm,
ec_mv,po_mv,pump_a,pump_b,pump_auto,pump_sample
```

True/false values are stored as 1/0 and missing values as empty cells. The files open directly in Excel or LibreOffice Calc. `temp_ok` is not a CSV column; it can be worked out from `temp`. The `data/` folder is not uploaded to GitHub.

3.4 Plant camera (MaixCAM + backend/vision.py)

The MaixCAM is a small computer with its own camera and screen. Its `hydro_cam` app starts at boot, shows a live preview (to aim the camera) and hands a fresh photo to the Pi over the USB cable.

- Every **10 s** `vision.py` takes a photo, checks it and publishes it to the dashboard.
- Every **15 min** (or with **Save photo now**) it also saves the photo to `data/vision/<date>/` and adds a row to `vision_log.csv`. Photos older than 30 days are deleted, except one per day.
- It also writes `data/vision/live/latest.jpg` so the dashboard can still show the photo if the live MQTT link is blocked.
- When a plant is in view it runs the growth-stage model (section 3.5) and adds `stage`, `stage_conf` and `size` to the result.

Health result	Rule (leaf colour check)
Healthy	Yellow/brown share of the leaf area is 8 % or less
Check leaves	Yellow/brown share above 8 %
Unhealthy	Yellow/brown share above 20 %
No plant in view	Less than 2 % of the photo is green (camera aim, focus or grow light)

```
{"ts": 1790161885, "image": "2026-09-23/07-11-25.jpg", "canopy_pct": 34.2,
 "yellow_pct": 3.1, "greenness": 0.41, "health": "healthy", "health_reasons": [],
 "stage": "vegetative", "stage_conf": 0.87, "size": "medium"}
```

3.5 Growth-stage AI model

`backend/train/train_stage.py` trains a small image model (MobileNetV3-Small, pretrained on ImageNet) on `dataset/lettuce_224/`, on the Pi 5 itself, in about 40 minutes. It compares two versions, a classifier and a regression model, with 5-fold cross-validation: the photos are split into 5 groups, and each group is tested by a model trained on the other four, so every photo is tested once by a model that never saw it. The better one is retrained on all photos and exported to `backend/models/stage.onnx`, which `vision.py` runs with ONNX Runtime from the `.venv` environment. The full report is `backend/models/stage_report.md`.

Result (88 photos, retrained 29 September 2026)	Value
Model chosen	A: classifier (B, regression, reached 76 %)
Test accuracy	86 % (always-guess-one-stage baselines: 34 % and 33 %)
Train accuracy	100 % (train–test gap 13 %: acceptable)
Mean absolute error	0.16 stage (RMSE 0.45)
Seedling / matured recall	97 % / 83 %
Vegetative recall (target 70 %)	79 % (23 of 29): target met
Old model (41 photos) on the 48 new photos it never saw	Old: 60 %, vegetative 29 %. New: 81 %, vegetative 79 %
Speed on the Pi 5	about 10 ms per photo

`stage` is seedling, vegetative, matured, or uncertain below 60 % confidence. `size` follows the stage (seedling = small, vegetative = medium, matured = large). The model only runs when the colour check finds a plant, because it knows only the three stages and would label anything as one of them.

3.6 Dashboard

A web page served by `backend/server.py` on port 8000 (page files in `frontend/`). Live values come from Mosquitto's websocket (port 9001); history comes from the session CSVs. Everything works offline (mqtt.js, uPlot and the DM Sans and Fraunces fonts are in `frontend/vendor/`).

Tab	What it shows
Live Monitoring	Header: Live dot, ESP32 and Camera shown as Connected (green) or Disconnected/Offline (red). Left: the camera photo in a square frame, the same centre crop the stage model sees, with health, growth stage, size, confidence and Save photo. Right: six small tiles (pH, EC, water, air 25–36 °C, humidity, pump mode + tank) with a meter and a green dot or LOW/HIGH/DRIFT tag; pump rows; Plant vision (canopy % with change vs yesterday, yellowing % with Normal under 8 %, green index 0–1); a small 24 h graph of the tile you click. Hover for details and raw values
Sensor History	pH (purple), EC (blue), air temperature (orange) and humidity (teal) as smooth lines with a soft fill, for 1 h, 24 h or 7 days; target band tinted, gaps hatched, hover shows values on all four; pump ON lanes and plant stage over time; Table view and CSV download

Tab	What it shows
Calibration	Step-by-step EC, pH and tank wizards. Capture buttons stay disabled until the reading is steady; the ESP32's replies show in the page; stored calibration and a hold-to-reset for pH

History API (server.py)	Returns
`/api/history?range=1h|24h|7d`	Readings on a fixed time grid (null = no data)
`/api/actuators/history?range=...`	Pump ON intervals, runs and on-time per pump
`/api/vision/history?range=...`	Rows from `data/vision/vision_log.csv`
`/api/export.csv?range=...`	Every session CSV row in the range, one file

3.7 Plant image dataset

`dataset/lettuce_growth_stages/` holds 88 lettuce photos sorted by growth stage (the folder name is the label). The originals come in different sizes and formats. `dataset/lettuce_224/` has the same photos all converted to the same size, **224×224 JPEG** (rotated upright, centre-cropped to a square, resized), with a `labels.csv` that adds the size. This copy is what the model trains on.

Stage	Folder	Photos	Size label
Seedling	`seedling/`	30	small
Vegetative	`vegetative/`	29	medium
Matured (ready to harvest)	`matured/`	29	large
Total		**88**	

The 28 September update added 48 new photos to the 40 kept from the first set, so the classes are now balanced. One photo was in the dataset twice, once as vegetative (`vegetative_012.jpg`) and once as matured (`matured_022.jpg`); both copies were removed because its stage is unclear. All photos are still from the internet: add labelled photos from the Hydro camera (`data/vision/`), then retrain.

4. What has been done

Area	Work	Status
ESP32 firmware	DHT22, EC and pH with calibration stored in EEPROM	Done
ESP32 firmware	HC-SR04 water level with tank calibration	Done
ESP32 firmware	Pump A/B auto control to EC 1.2–1.8 and pH 5.5–6.5; manual commands	Done
ESP32 firmware	Always-on sampling pump (GPIO27) and tank-low stop for all pumps	Done
ESP32 firmware	Air temperature target 25–36 °C, reported as `temp_ok`	Done
Hardware	1-channel relay replaced by a second 2-channel relay (sampling pump on relay 2 IN1; IN2 spare); one shared `RELAY_ACTIVE_LOW`	Done
Testing	Relay test sketch: relay 1 switched Pump A and Pump B correctly	Done
Testing	Sampling pump on relay 2: check with `sampleoff` / `sampleon`	To do
Raspberry Pi	Serial → MQTT bridge with auto-reconnect and fixed /dev/esp32 name	Done
Raspberry Pi	Mosquitto broker (local TCP + websocket for the dashboard)	Done
Data logging	CSV log per ESP32 connection, including pump columns	Done
Camera	MaixCAM app: auto-start, preview, photo server over USB	Done
Camera	vision.py: live photos, leaf colour health check, photo archive	Done
Camera AI	Growth-stage dataset: 88 photos, balanced (30 / 29 / 29), plus a 224×224 copy with size labels	Done
Camera AI	Growth-stage + size model retrained on 88 photos (86 % test accuracy, vegetative recall 79 %)	Done
Dashboard	Live Monitoring, Sensor History and Calibration tabs; history API and CSV export	Done
Repository	Reorganised into `frontend/`, `backend/`, `firmware/`	Done
Camera AI	Label photos from the Hydro camera and add them to the dataset, then retrain	To do
Sensors	DS18B20 water temperature for EC compensation	To do
Safety	Password/ACL on MQTT; maximum run time for Pumps A/B	To do

Development history (GitHub commits)

Commit	What changed
6214b11	Initial commit: ESP32 firmware, serial bridge, README
5b9a0b8 · 97d15b4	Full README (wiring, calibration, troubleshooting); systemd, Mosquitto and udev configs
9de0bf0 · 27b1f4d	Sensor prototyping sketches, circuit diagram, relay test, HC-SR04 in the firmware
07c09f3 → 6a15904	Pump A/B relay control, tank-low stop, EC/pH target control, always-on sampling pump
bd111f6	CSV session logs with pump state
68434be	Dashboard design plan
2856219	MaixCAM plant camera with leaf colour health check
3272c0d · a066e70	First live dashboard page, with an HTTP fallback for the camera photo
02f4b24 · 8e573f3	System guide (PDF), version 1
24d1ede · 84a2ea7	Lettuce growth-stage dataset (41 photos) and its 224×224 copy with size labels
761e8d3	Growth-stage model, Graphs and Calibration tabs, history server
5094bcc · d537789	Repository reorganised into frontend/, backend/ and firmware/
ff034c4	Air temperature target 25–36 °C (`temp_ok`)
e59755a	1-channel relay replaced by a second 2-channel relay
d131785	New lettuce dataset: 90 photos (30 per stage)
f962ca6	System guide version 2
5dfbf3f	Duplicate photo removed (88 photos), growth-stage model retrained, system guide version 3
827a7ce	Out-of-range pH readings ignored in history; graph axes capped
42c3365	Minimalist dashboard: new header and tab names, square camera frame, Plant vision card, coloured sensor graphs, DM Sans + Fraunces
(this version)	System guide version 4

Current status

Part	State
ESP32 + bridge	Running · sending data with pump columns (log #0007, 23 Sep)
Relays	Updated · two 2-channel modules; firmware updated. Re-flash `Full_code.ino` and confirm the sampling pump runs on relay 2 IN1
Mosquitto	Running
MaixCAM	Running · photos reach the Pi; aim the camera at the plants
Stage model	Updated 29 September 2026 · retrained on 88 photos; restart `hydro-vision` to load it
vision.py / dashboard	Check · install the services (section 5.2) so they survive a reboot; vision.py needs `.venv` for the stage model
pH probe	Check · read 19.71 on 23 Sep, which means disconnected or not calibrated
Water level	Check · no echo from the HC-SR04 on 23 Sep (level empty in the log)
EC	Check · `ec_low` on 23 Sep (probe in plain water or dry)

5. How to run the system

5.1 First-time setup

Upload `firmware/Full_code/Full_code.ino` to the ESP32 with the Arduino IDE (board: ESP32 Dev Module; libraries: DHT sensor library and Adafruit Unified Sensor; the patched DFRobot_EC files are in the same folder). Re-upload it after the relay change so the ESP32 runs the version with the shared relay setting. Then on the Pi:

```
sudo apt install mosquitto mosquitto-clients python3-serial python3-paho-mqtt python3-pil
cd ~/hydro
sudo cp backend/mosquitto/local.conf /etc/mosquitto/conf.d/
sudo cp backend/udev/99-esp32.rules /etc/udev/rules.d/
sudo udevadm control --reload-rules && sudo udevadm trigger
sudo usermod -aG dialout $USER            # then reboot
sudo systemctl restart mosquitto
```

Python environment for the growth-stage model (one time):

```
cd ~/hydro
python3 -m venv .venv
.venv/bin/pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
.venv/bin/pip install -r backend/train/requirements.txt
```

Install the camera app on the MaixCAM (one time):

```
scp -r ~/hydro/maixcam/hydro_cam root@10.122.173.1:/maixapp/apps/
ssh root@10.122.173.1 'echo hydro_cam > /maixapp/auto_start.txt && reboot'
ssh root@10.122.173.1 passwd          # change the factory password
```

5.2 Install the three services (start at every boot)

```
cd ~/hydro
sudo cp backend/systemd/hydro-bridge.service backend/systemd/hydro-vision.service \
        backend/systemd/hydro-dashboard.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now hydro-bridge hydro-vision hydro-dashboard
```

If vision.py or the dashboard are still running from a manual start, stop them first: `pkill -f vision.py; pkill -f server.py`

5.3 Daily use

- Power on the Pi with the ESP32 and the MaixCAM plugged in by USB. All services start by themselves.
- Open the dashboard: `http://localhost:8000` on the Pi, or `http://<pi-ip>:8000` from a phone on the same WiFi (find the IP with `hostname -I`).
- Check the three dots at the top right: MQTT, ESP32 and Camera should be green.
- Check the pump cards: Sampling should show ON (relay 2) unless the tank is low or `sampleoff` was sent.
- Aim the camera at the lettuce using the preview on the MaixCAM screen. Take photos under the grow light.

5.4 Useful commands

Command	What it does
`systemctl status hydro-bridge hydro-vision hydro-dashboard`	Are the services running?
`journalctl -u hydro-bridge -f`	Follow the bridge log (ESP32 messages, new CSV files)
`journalctl -u hydro-vision -f`	Follow the camera log (saved photos, health and stage results)
`mosquitto_sub -t 'hydro/#' -v`	Watch everything on MQTT (the image topic prints binary)
`mosquitto_pub -t hydro/cmd -m status`	Ask the ESP32 for its calibration, pump and sampling pump state
`mosquitto_pub -t hydro/cmd -m pumpauto`	Put Pumps A and B back in automatic mode
`mosquitto_pub -t hydro/cmd -m sampleoff`	Stop the sampling pump (relay 2); `sampleon` starts it
`mosquitto_pub -t hydro/vision/cmd -m snap`	Save a camera photo now
`~/hydro/.venv/bin/python ~/hydro/backend/vision.py --once`	Test the camera: one photo, print the result
`.venv/bin/python backend/vision.py --stage photo.jpg`	Run the stage model on a photo file
`.venv/bin/python backend/train/train_stage.py`	Retrain the stage model (about 40 min), then restart hydro-vision
`.venv/bin/python docs/guide/build_guide.py`	Rebuild this guide (PDF)
`ls -lt ~/hydro/data/logs | head`	Newest sensor CSV logs
`curl http://10.122.173.1:8080/status`	Is the camera app answering?
`sudo systemctl restart hydro-bridge`	Restart the bridge (starts a new CSV log)

6. Calibration

Use the dashboard's Calibration tab, the Arduino Serial Monitor (115200, Newline) or `mosquitto_pub -t hydro/cmd -m <command>`. All values are stored in the ESP32 and survive reboots. Check them with `status`.

Sensor	Steps	Expected
EC	`enterec` → probe in 1413 µS/cm buffer → `calec` → probe in 12.88 mS/cm buffer → `calec` → `exitec`	~230 mV in 1413, ~2100 mV in 12.88; K between 0.5 and 1.5
pH	Probe in pH 7 buffer, wait 1–2 min → `ph7` → rinse → pH 4 buffer, wait → `ph4` (`phreset` = defaults)	Slope about 180 mV/pH
Tank level	Tank empty → `tankempty` → fill to MAX → `tankfull` (`tankreset` = defaults)	Empty and full at least 3 cm apart; sensor at least 2 cm above MAX

The tank-low pump stop (all pumps on both relays) only works after the tank is calibrated. Before the first pH calibration, zero the module: short the BNC and turn the left potentiometer until `po_mv` ≈ 2500. After use, rinse the pH probe and put the cap back on with its storage solution; the glass bulb must never dry out.

Lettuce target ranges

Parameter	Target	Set in
EC	1.2–1.8 mS/cm (seedlings 0.5–1.0)	`EC_MIN` / `EC_MAX`: pumps act on it
pH	5.5–6.5	`PH_MIN` / `PH_MAX`: pumps act on it
Air temperature	25–36 °C	`TEMP_MIN` / `TEMP_MAX`: reported only (`temp_ok`)
Humidity	50–75 % (placeholder)	Dashboard band only
Water level	above 20 %	`LOW_LEVEL_PCT`: all pumps stop below it

The dashboard bands must match the firmware values; both are listed at the top of `Full_code.ino` and in `frontend/index.html`.

7. Troubleshooting

Problem	Fix
`/dev/esp32` missing	Check `lsusb`, re-plug the cable, reload the udev rules
No data on `hydro/sensors`	`journalctl -u hydro-bridge -f`; close the Arduino Serial Monitor (only one program can use the port)
Sampling pump does not run	Check `pump_sample` in the JSON. If it is true but the pump is off: relay 2 IN1 on GPIO27, relay 2 VCC on 5 V, GND shared with the ESP32, pump on the CH1 contacts (not CH2). If it is false: tank low, or `sampleoff` was sent (send `sampleon`)
A relay is ON when it should be OFF	The module is active HIGH: set `RELAY_ACTIVE_LOW` to `false` and re-upload (it applies to both relays)
Pumps A/B never switch	Relay 1 IN1/IN2 on GPIO25/26; test with `firmware/Relay_Test/`; check the tank is not low
pH stuck (e.g. 19.71)	Probe/BNC disconnected or not calibrated; check wiring, then calibrate
`ec_low: true`	Solution below about 0.6 mS/cm (normal for plain water)
Water level empty	HC-SR04 gets no echo: check wiring, the ECHO divider, and that it faces the water
`temp_ok: null`	No DHT22 reading: check the DHT22 wiring and pull-up resistor
Dashboard shows old data	Hard refresh (Ctrl+Shift+R). If the header says "Live link down", the browser can't reach MQTT on port 9001: restart the browser or turn off HTTPS-Only for the site. The camera photo still updates via HTTP
Stage always empty	vision.py is running without ONNX Runtime: start it from `.venv` (the service does this), and check `backend/models/stage.onnx` exists
Stage looks wrong after retraining	Restart `hydro-vision` so it loads the new model; the old one is kept as `backend/models/stage_old.onnx`
`hydro/vision/status` offline	`ping 10.122.173.1`; check the USB-C cable and that the MaixCAM shows the Hydro Cam preview
MaixCAM freezes / SSH hangs	Two programs used the camera. Reboot the MaixCAM. Never run camera scripts over SSH while its menu is running
Health says "No plant in view"	Aim the camera at the plants, check focus, switch on the grow light

8. Known issues and next steps

Safety items first

- **Pumps can dose without limit.** With a bad pH reading (such as 19.71) or a dry EC probe, the firmware sees "out of target" and runs Pumps A and B continuously. Add a maximum run time, or dose in short pulses (e.g. 2 s ON, wait 30–60 s, re-check). Until then, keep the nutrient pumps unplugged when the probes are not in solution.
- **MQTT has no password.** Anyone on the WiFi can send pump or reset commands to `hydro/cmd` (the dashboard's Calibration tab uses the same topic). Add a Mosquitto password and an access list before using the dashboard from other devices.
- **MaixCAM factory password.** Change it with `passwd` (section 5.1).
- **Relay wiring.** Keep relay contacts and any mains wiring in a closed box away from water.

Roadmap

- Confirm the sampling pump on relay 2 after re-flashing the firmware.
- Grow-light control on the spare relay channel (relay 2 IN2): pick a free GPIO, add a schedule in the firmware and a `light` field in the JSON.
- Add labelled photos from the Hydro camera (`data/vision/`) to the growth-stage dataset and retrain, so the model learns the real rack view, not only internet photos.
- DS18B20 water temperature for EC temperature compensation.
- RTC battery for correct offline time.